Dev Vyas Portfolio - Complete Reference Guide

Site: dev-vyas-portfolio.vercel.app **Repository:** github.com/vyasgraphics/dev-vyas-portfolio **Owner:** Dev Vyas, Product Designer and UX Researcher **Document version:** August 2026 (revision 7)

About this document

This is a complete written reference for the portfolio site: what it contains, how it is put together, why certain decisions were made, and what to be careful about when changing things.

It is written for two readers. The first is you, six months from now, coming back to make a change and needing to remember why something works the way it does. The second is anyone else who might need to pick the project up - a collaborator, or a developer helping with a future feature.

Where a decision was unusual or hard-won, the reasoning is recorded rather than just the outcome. A surprising number of the fixes in this project came from problems that looked like one thing and turned out to be another, and those are the details most easily lost.

The narrative arc (revision 4)

As of revision 4 the homepage is written to read as a story, not a set of labelled panels. The section tags are deliberately phrased as chapters that answer three questions a recruiter is asking, in order:

- **Who I am** - the Intro headline ("Designing interfaces backed by research, not guesswork"), the "Who I Am" About section, and the stats (4+ years, 25+ users researched and tested).
- **How I work** - "Selected Work" as the proof, then "How I Work" (the Skills section, reframed with the heading "Research first, pixels second"), "The Path Here" (Background) and "What I Build With" (Tools).
- **Why I'm the fit** - "How I Think" (Writing) and the closing "Why I'm the Fit" line above the contact form: "A designer who researches, a researcher who ships, and someone who uses AI to move faster without letting it make the calls."

Each case study reinforces the same arc with an outcome banner directly under its H1 (a green left-border strip with the single most important result), so the "so what" lands within the first few seconds of a scan.

If you reword any section tag, keep the chapter logic intact - the labels are load-bearing for the story, not decorative. The sidebar navigation labels (nav.ts) are intentionally kept short and functional (Work, About, Skills, Background, Tools, Writing, Contact) and do not need to mirror the narrative tags.

Long-form page margins (revision 5)

On a wide desktop viewport, a fixed reading-width column centred in the page leaves large empty margins on both sides - visible as dead space on anything wider than about 1400px. The reading column stays capped at 780px (matches the case study width, unchanged from the original template) for line-length readability; the fix instead gives the right margin a job:

- `SectionNav` on the right (`right: 28px` , desktop-only from 1180px) - the dot-rail in-page navigation component, now wired into all six long-form pages (three case studies, three blog posts). Each page defines a `SECTIONS` array of `{id, label}` matched to its H2 headings (which carry matching `id` and `scrollMarginTop` attributes). Blog posts pass their array to `BlogPostLayout` via an optional `sections` prop, which renders `SectionNav` internally; case study pages render it directly, as they always have.

A matching left-margin element (`ReadingProgress` , a scroll-tracking fill) was built and shipped in revision 5/6, then removed at Dev's request in revision 7 - the general direction (functional margins on wide viewports) is still worth having, but two green vertical elements on the same page - the new rail and the site's own green-styled native scrollbar thumb (`body::-webkit-scrollbar-thumb` in styles.css, pre-existing, unrelated) - read as confusingly similar at a glance. If a left-margin element is wanted again, it should look visually distinct from the browser's own scrollbar (e.g. positioned away from the very edge, or a different visual treatment entirely) rather than another thin vertical green line.

Two readability fixes landed alongside the narrative pass:

- **Profile card greeting.** The rotating line reads "Hey, I'm ..." cycling through "Dev Vyas", "a Product Designer", "a UX Researcher", "a Graphic Designer". The articles ("a") matter: without them the roles read as "Hey, I'm Product Designer", which is ungrammatical. The rotation values live in `rotatingNames` in `profile.ts` , used only by `UserSidebar.tsx` . The greeting also carries a `text-shadow` so it stays legible over any part of the background photo at every point in the clip animation. Behind it, a dark scrim covers the bottom portion of the photo so the greeting and bio never blend into the image: on mobile via `.user-image::before` (78% height, near-solid black base), and on desktop via `.user-image .image::after` (72% height) - the desktop one was added because `.user-info` is absolutely positioned over the photo at every width, but the original scrim was mobile-only, so wide screens had text sitting straight on the image. `.user-info` carries `z-index: 2` so it always paints above the scrim. The vertical "Available Sep 2026" pill (`.meta-left`) is anchored at `bottom: 46%` rather than centred at `top: 50%` , so it sits in the photo area above the greeting and never overlaps the "Hey, I'm ..." line as the

greeting length changes. All of these are defined in both `styles.css` and `scss/component/elements/_section.scss` - edit both or the SCSS recompile silently reverts the CSS.

- **Work section tag.** "Selected Work" is no longer `position: sticky`. It was pinned at `top: 72px` when it was the lone label above the cards, but the new connector paragraph beneath it scrolled up under the pinned pill and overlapped it. The three work cards keep their own sticky scroll behaviour; only the tag pill was un-pinned. Removed in both `styles.css` and `_section.scss`.

Part 1: The site at a glance

What it is

A single-page portfolio homepage with six supporting pages: three long-form case studies and three written articles. The homepage carries the introduction, work highlights, background, skills, tools and contact form. The supporting pages go deep on individual pieces of work.

Structure

Route	Purpose
`/`	Homepage: intro, work, about, skills, background, tools, writing, contact
`/work/dissertation`	Case study: distraction resistance and complex interfaces
`/work/move-app`	Case study: Move, a university exercise app
`/work/vyas-graphics`	Case study: brand identity and sports media
`/blog/ai-in-ux-research`	Article: where AI fits into UX research
`/blog/dissertation-notes`	Article: notes from running the dissertation study
`/blog/building-move-app`	Article: building Move end to end

Plus two generated files that exist for search engines rather than people: `/sitemap.xml` and `/robots.txt`.

Scale

Roughly 11,000 lines of TypeScript and TSX across around 60 components, plus an 8,900 line stylesheet inherited and heavily modified from the original template.

Part 2: Technology

The stack

Layer	Choice	Version
Framework	Next.js (App Router)	16.2.6
UI library	React	19.2.4
Language	TypeScript	5.x
Styling	Bootstrap 5 plus custom CSS and SCSS	5.3.8
Scroll animation	GSAP with ScrollTrigger, SplitText, ScrollToPlugin	3.15.0
Smooth scrolling	Lenis	1.3.23
Scroll-linked motion	Motion (Framer Motion)	13.0.0
Marquee	react-fast-marquee	1.6.5
Carousel	Swiper	12.1.4
Contact email	Resend	6.18.1
Theme handling	next-themes	0.4.6
Hosting	Vercel	-

A note on styled-components

Several components on this site were adapted from published snippets that ship as styled-components. None of them brought the dependency with them.

The reasoning: this project has no styled-components anywhere, and adding a runtime CSS-in-JS library to a Next.js App Router build means wiring up a style registry for server rendering and accepting the risk of a flash of unstyled content on first paint. The codebase already has a proven pattern for scoped component styling, which is plain CSS classes in the main stylesheet, and every adapted component uses that instead. The visual result is identical and there is one less thing that can break at build time.

Why two animation libraries

This looks like duplication but is not. GSAP with ScrollTrigger handles the majority of the site: text reveals, the work section's sticky card behaviour, counters, the scribble drawing. It was inherited from the template and is well suited to timeline-based sequencing.

Motion (Framer Motion) was added later for one specific job: the 3D welcome reveal on the homepage. That effect needs a value that maps continuously to scroll position rather than a timeline that fires at thresholds, and Motion's `useScroll` and `useTransform` express that far more cleanly than the GSAP equivalent would.

Both listen to native scroll events, and Lenis (in root mode) fires those events synchronously with `window.scrollY`, so the two coexist without any special integration work.

Part 3: Project structure

```
dev-nextjs/
├── public/
│   └── assets/
│       ├── css/styles.css          Main stylesheet (8,856 lines)
│       ├── scss/                   SCSS source for the same styles
│       ├── fonts/
│       │   ├── inter/              Body typeface
│       │   └── tomorrow/           Welcome screen typeface, self-hosted
│       ├── icon/icomoon/           Icon font
│       └── images/
│           ├── section/            Homepage and shared imagery
│           ├── vyas-graphics/      Case study assets (52 files, 6 videos)
│           ├── item/               Timeline icons
│           └── logo/               Logos and favicons
└── src/
    ├── app/                        Routes (App Router)
    │   ├── layout.tsx              Root layout, site-wide metadata
    │   ├── page.tsx                Homepage entry
    │   ├── sitemap.ts              Generated sitemap
    │   ├── robots.ts               Generated robots file
    │   ├── opengraph-image.tsx     Generated social share image
    │   ├── api/contact/route.ts    Contact form handler
    │   ├── work/                   Three case study pages
    │   └── blog/                   Three article pages
    ├── components/
    │   ├── sections/               Nine homepage sections
    │   ├── wireframes/             Interactive wireframes for case studies
    │   └── [components]            Around 40 shared components
    ├── data/                       Content, separated from presentation
    ├── hooks/                      Custom React hooks
    └── lib/smoothScroll.ts         Scrolling helper used site-wide
```

The thinking behind the layout

Content lives in `src/data` as typed TypeScript objects, not hardcoded into components. That means updating a job title, a project description or a blog excerpt is a one-line change in a data file rather than hunting through markup. It also means the homepage blog card and the blog post header cannot drift out of sync, because both read from the same source.

Wireframes have their own folder because they are a genuinely distinct category: they are not reusable interface pieces, they are illustrative recreations of screens from research work, built to be embedded in case studies and articles.

Part 4: The homepage

The homepage is assembled in `HomeShell.tsx`, which stacks the chrome, the welcome reveal and then the nine content sections in order.

The loader

Before anything else, a brief loading ring: a circle with opposite quarters knocked out, which swaps which pair of arcs is visible halfway through each turn so it reads as a ring folding rather than simply spinning. Drawn in the site green, with a darker green appearing at the midpoint swap.

One thing to know: the loader is dismissed after 400 milliseconds against a one second animation cycle, so in practice a visitor sees about a third of a turn. That is deliberate, since a loader that lingers is worse than one barely noticed, but it does mean the animation is never seen in full. If it ever needs to be, the timing lives in `Preloader.tsx`.

The welcome reveal

The first thing a visitor sees is a full-screen introduction: the words "DEV VYAS" assembling themselves in 3D space as you scroll, with "Welcomes you" appearing beneath and a "Scroll to enter" prompt at the bottom.

How it works. A scroll track (160vh, reduced from 200vh in revision 4 so the homepage's substance appears sooner) contains a sticky stage pinned to the viewport. As the track scrolls past underneath, scroll progress drives each letter's rotation, depth and horizontal offset. Because the animation is keyed to scroll progress (0 to 1) rather than pixels, shortening the track does not clip or rush the sequence - the whole reveal simply resolves over less scroll distance. Letters further from the centre of the word start more dramatically displaced, so the word closes in from both ends rather than every letter moving identically.

The typeface. The welcome text uses Tomorrow at weight 700, a geometric typeface chosen to contrast with the body text. The ExtraBold (800) file is still bundled but no longer referenced by any rule; browsers only download weights that are actually matched, so it costs nothing at runtime and keeps the heavier option available. It is self-hosted from the site's own domain rather than loaded from Google Fonts. This matters: the earlier version loaded it from Google's CDN and it silently failed to render for anyone using a browser that blocks third-party font requests, Brave among them. Self-hosting removes that dependency completely. Only two weights are bundled (700 and 800), converted to WOFF2, totalling around 32KB.

The chrome fade. The navigation sidebar and profile card are hidden while the welcome screen is showing, and fade in as it fades out. Both are driven from the same scroll value, so they crossfade in step rather than one appearing after a gap. While hidden they are also unclickable, not merely invisible.

Reduced motion. If a visitor has reduced motion enabled at the system level, the whole thing collapses to a short static section rather than two viewport-heights of scrolling for an effect they will not see.

The nine sections

Section	Anchor	On-screen tag (rev 4)	What it holds
Intro	`#home`	(headline)	Headline "backed by research, not guesswork", rotating job titles, availability, stats (4+ years, 25+ users), discipline marquee
Work	`#work`	Selected Work	Three project cards with sticky scroll behaviour, plus a one-line connector
About	`#about`	Who I Am	Positioning statement and certifications
Skills	`#skill`	How I Work	Heading "Research first, pixels second"; four expandable skill areas
Background	`#education`	The Path Here	Combined education and work timeline
Tools	`#tech`	What I Build With	Seven categories of software
Writing	`#blog`	How I Think	Three article cards
Contact	`#contact`	Why I'm the Fit	Closing-argument line plus contact form
Footer	`#footer`	(tagline)	"Research tells me what to build. Craft makes people want to use it."

The discipline marquee

Below the availability line, eight disciplines scroll past continuously: UI/UX Design, User Research, AI-Augmented Design, Brand Identity, Motion Design, Usability Testing, Wireframing and Print Design. The order deliberately mirrors the Skills section.

There is a fix worth knowing about here. The pills are spaced by a margin applied to each individual pill, not by a flexbox gap on their container. The reason: the marquee library wraps each repeated batch of pills in its own container, and a gap only applies within a single container, so the seam between batches ended up narrower than every other gap. Applying the spacing per pill keeps it identical everywhere. There was also a leftover rule from the original template setting a fixed margin on the same class name, which had to be accounted for.

The tools section

Seven categories, each in its own card that stays dark until you flip its switch. Ordered to follow the actual shape of the work rather than alphabetically:

1. **UI / UX** - Figma, Adobe XD, Framer
2. **AI for UX** - Claude, Gemini, Gemini Notebook
3. **Quantitative Data Analysis** - Python, Jupyter Notebook, Google Colab
4. **No-Code / Low-Code / Vibe Code** - Claude Code, Lovable, Bolt
5. **Web Development** - HTML5, CSS3, JavaScript, React, Next.js
6. **Product / Web Deployment** - GitHub, Vercel, Firebase, Netlify
7. **Graphics** - Illustrator, Photoshop, After Effects, InDesign, Premiere Pro

The order is design first, then the AI layer sitting on top of both design and development, then the quantitative side of research, then coding, then shipping what was built, then the graphics and motion work that finishes a piece. Do not reorder these without good reason.

The illumine cards

Categories sit inline in a wrapping grid: a fluid auto-fit layout on desktop, two columns on tablet, one on phones. As seven full-width rows the section ran about three screens for what is, in substance, a list of software; as a grid it is one screen and can be taken in at a glance.

Each card rests desaturated and lights on hover, casting a cone down over its logos and bringing them to full colour. This replaced an explicit on/off switch per card. The switch worked, but it asked the visitor to operate something before the section would show them anything, and on a portfolio being skimmed in under a minute nothing should need to be operated to be legible. Touch devices have no hover, so the same states are driven by a tap there, and a tap elsewhere puts the card back to rest.

Adapted from a portrait "luminous card" reference, which needed three changes to work here. The shape went from a fixed portrait card to a wide one whose height varies with how many tools a category holds, so every fixed measurement became a percentage or a clamp, and the light source moved from the centre to just under the top edge. The colour went from white to the site green. The switch became a real button with `role="switch"` and `aria-checked` rather than a hidden checkbox, so it is announced properly and works from the keyboard, and it carries the same press feedback as every other control here.

One deliberate restraint: the resting state is desaturated rather than dim. At the opacity that looked most dramatic, the tool names fell to roughly 3.5:1 contrast, under the 4.5:1 that body text needs. Those names are real content and have to be readable without hovering, so losing the colour carries the resting signal instead, and the colour returning is what makes the hover feel like it did something.

Logo colours

Most tool logos keep their brand colours, which read fine against the dark background. The exceptions are the marks that are pure black in brand form and would otherwise be invisible: Next.js, GitHub and Vercel are all drawn in white.

The background timeline

Six entries, education and work interleaved chronologically:

- Student Content Creator, University of York (Oct 2025 to present)
- MSc Human-Centred Interactive Technologies, University of York (2025 to present)
- Graphic and UI Designer, Panchal Softwares (Aug to Dec 2024)
- Design Lead and Mentor, Google Developer Student Club (Jan 2023 to Jan 2024)
- BE Computer Science and Engineering, Gujarat Technological University (2020 to 2024)
- UI/UX and Visual Consultant, non-profit organisations (Jun 2020 to Apr 2023)

Certifications

Four, listed in the About section: the Google UX Design Professional Certificate (Coursera, 2025), Adobe Graphic Designer (Coursera, 2025), Advanced Certification in Data Science with Specialisation in Business Intelligence and Data Analytics (IIIT Bangalore, 2025), and Specialisation in Animation and Multimedia (Frameboxx, 2020).

Part 5: The case studies

Every case study opens with an **outcome banner** directly under its H1 (added in revision 4): a single bolded line with a green left-border strip, carrying the one result that matters most. It exists so a recruiter scanning for under a minute gets the "so what" before the tags, not after three paragraphs. Dissertation leads on the uneven time cost of clutter; Move on testers finally finishing a session; Vyas Graphics reframes it as scope ("20+ brand identities... all shipped") since it is a body of work rather than a single finding.

Dissertation: distraction resistance and complex interfaces

Route: `/work/dissertation`

MSc capstone research asking whether a person's intrinsic ability to resist distraction predicts how well they perform on a visually cluttered interface. Run as a quantitative online study via Prolific, measuring visual working memory distraction resistance against a real-world complex interface, with reading speed as a control variable.

Sections: The Question, The Study, The Key Finding, What It Means, What's Next.

The page uses two custom wireframe components rather than screenshots. `WireframeNewsTask` recreates the cluttered news search task participants completed, and `WireframeCirclesTest` is a static three-panel diagram explaining the distraction-resistance measure across its no-distraction, encoding and delay stages. There is also a custom SVG chart showing the interaction effect.

Note on content: the written findings are deliberately kept high level, without exact statistics or p-values, until the dissertation is formally submitted.

Move: university exercise app

Route: `/work/move-app`

A full human-centred design lifecycle project asking how to get students exercising despite time pressure and fear of judgement. User research with 17 participants, personas, Figma prototyping, think-aloud testing with 8 users, ending in an A/B study proposal. A university module project with collaborators Haokai, Lanqing and Yechen.

Sections: The Problem, Who It's For, The Screens, Key Finding, What's Next.

Five interactive wireframes let a reader try the concepts rather than just read about them: Smart Input, Quiet Mode, See Before You Go, Gap Finder and Crowd Filter.

Vyas Graphics: brand identity and sports media

Route: `/work/vyas-graphics`

Self-directed brand and motion work spanning four years: logo design, animated logo reveals, social media, print and a full sports media campaign for the ICC T20 World Cup 2026 and IPL 2026.

Six sections, each in its own bordered card with a green tag: Brand Identity, Motion, Social Media, Print, Sports Media and Flipbooks.

This is the heaviest page on the site: 52 image and video assets, including six transcoded H.264 clips. Several custom components exist mainly for this page:

- **GlassDeck** - a fanned deck of cards that spreads open on hover, or on tap on touch devices
- **PlayableStill** - a poster image that becomes a playing video
- **VideoPlayer** - a custom player with a control bar
- **BeforeAfterSlider** - a draggable comparison between two images
- **NotificationStory** - three linked cards telling the story behind the flipbook work
- **FlipBookCard** - a portfolio cover that turns over to show its back cover
- **InstagramButton** - a circular button that fills with the Instagram gradient and raises the handle above itself

---

The flipbooks

Four editions of the same portfolio, rebuilt year on year: 2022, the LinkedIn upload, 2024, and the current 2026 edition. Each is shown as a card that turns over to reveal its back cover, with a link out to the live page-turning version.

The flip is driven by state as well as hover. Hover alone, which is what the original reference used, does nothing at all on a phone. The link sits below the card rather than wrapping it, because wrapping would make a single tap both flip the card and open a new tab.

Part 6: The writing

Three articles, all fully written and live:

How AI Actually Fits Into a UX Research Workflow (And Where It Still Can't Replace You) - 7 August 2026, 8 minute read. An honest account of where AI genuinely helps in research and where it does not, avoiding both the "AI replaces researchers" and "AI has no place here" positions.

What My Dissertation Taught Me About Complex Interfaces - 8 August 2026, 7 minute read. Behind the scenes on designing the distraction-resistance study, including the data collection pipeline across Prolific, Qualtrics, the circles test and the news search task.

From Lo-Fi to Live: Building Move App End to End - 7 August 2026, 9 minute read. The full HCD lifecycle behind Move, including

rejected concepts and a mid-project redesign.

All three share `BlogPostLayout`, which pulls the title, date, reading time, tag and hero image from `blog.ts` by slug. Update the data file and both the homepage card and the article header stay in step automatically.

Part 7: Components

Layout and navigation

Component	Job
`HomeShell`	Assembles the entire homepage
`DesktopSidebar`	Fixed dot navigation on the right
`MobileMenu`	Mobile navigation
`UserSidebar`	Profile card
`HeaderClock`	Date and time display
`SectionNav`	In-page navigation - dot rail on all three case studies and all three blog posts
`BackLink`	"Back to Work" and "Back to Writing" links
`BackToTop`	Floating scroll-to-top button
`Preloader`	Initial load screen
`BodyBackground`	Background layer

Motion and reveal

Component	Job
`WelcomeReveal`	The 3D scroll introduction
`ScrollReveal`	Fades content in as it enters view
`StaggerReveal`	Same, with children offset in sequence
`AutoRepeatMarquee`	Continuous horizontal scrolling
`TiltCard`	Tilts towards the pointer or device orientation
`ScaleToFit`	Scales fixed-size content down to fit narrow screens

Media

Component	Job
`GlassDeck`	Fanned card deck
`PlayableStill`	Poster that becomes a video
`VideoPlayer`	Custom video controls
`BeforeAfterSlider`	Draggable image comparison
`ImageSwitch`	Swaps asset by theme
`LogoMarkTile`	Logo display tile

Case study specific

`PersonaCard`, `ScenarioCard`, `ClaimsTable`, `RejectedConceptCard`, `PipelineDiagram`, `MergePipelineDiagram`, `DistractionInteractionChart`, `NotificationStory`, `SectionBox`, `SectionDivider`.

Wireframes

`WireframeNewsTask`, `WireframeCirclesTest`, `WireframeSmartInput`, `WireframeQuietMode`, `WireframeSeeBeforeYouGo`, `WireframeGapFinder`, `WireframeCrowdFilter`, plus `PhoneFrame` and `BrowserFrame` as containers.

Files kept but not currently rendered

`BentoIntro.tsx` and `BrandSlider.tsx` are both on disk but not used anywhere. BentoIntro was built as a tile grid for the Vyas Graphics page and then removed as redundant against the section navigation. BrandSlider came from the original template. Both are harmless, and are kept in case they are wanted later.

Part 8: Custom hooks

Hook	Job
`useScrollAnimations`	The main animation engine: text reveals, the work section, counters, the scribble, the headline highlight
`SmoothScroll`	Wraps the app in Lenis
`useUrlHashSync`	Keeps the address bar hash in step with the section in view
`useCrossRouteBackNav`	Restores position when returning from a sub-page
`useResetScrollOnForwardNav`	Starts sub-pages at the top
`useBodyThemeClass`	Locks the site to the dark theme
`useClock`	Drives the header clock
`useHeadlineRotate`	Rotates the job titles in the intro
`useInfiniteSlide`	Continuous slide effects
`useDeviceTilt`	Device orientation for tilt effects
`BootstrapClient`	Loads Bootstrap's JavaScript

Part 9: Styling and theming

The theme

The site is permanently locked to a single dark theme, internally called `dark-v3` or "Forest Shadow". The original template shipped a theme picker with seven variants; that was removed deliberately. The picker component and its data file have both been deleted, and `useBodyThemeClass` clears any leftover preference from browser storage on load.

Core colours

Purpose	Value
Background	`#0A0A0A`
Primary accent	`#00DE51`
Secondary accent	`#00C853`
Body text	White at varying opacity

The background value is worth a note. It is defined in two places: the compiled `styles.css` and its SCSS source in `_themes.scss`. Both must be changed together. An earlier attempt to change it only in the CSS appeared to do nothing, because the SCSS source is also compiled into the build and was overriding it.

Typography

The heading scale was pulled down from the template's original sizes, which topped out at 60px for h1 and 52px for h2. Those suited a template demo where the headline was the content; here the headings sit above material someone is trying to scan quickly, and oversized type pushed that material further down the page. Each level came down by roughly a fifth, keeping the same rhythm between levels.

Section headings are sized on the `.s-title` class rather than inheriting from the tag. That matters: when the heading hierarchy was corrected for accessibility, those headings went from h4 to h2, which was right semantically but silently made every one a step larger, because size was tied to the tag. Setting size on the class keeps the two independent.

Inter for body text and headings, EB Garamond for the italic footer tagline, Tomorrow for the welcome screen only, and an icomoon icon font for interface icons.

Interaction feedback

Every interactive element has a consistent press response: it grows very slightly on hover (on devices with a real pointer only) and compresses on press, with a soft inset shadow.

Hover effects are wrapped in a `hover: hover` media query so touch devices do not get stuck in a hover state after a tap. There is no white ring on press: an earlier version had one and it read as a white box flashing on every click.

Three micro-interactions were added in revision 4:

- **Blog card images** rest at `grayscale(35%)` and animate to full colour with a gentle `scale(1.04)` zoom on hover (clipped by the card's `overflow: hidden`). Hover is gated behind `hover: hover`; a top-level `:active` rule gives touch users the same de-grayscale on tap, so nothing is inert on a phone. The image carries a `.blog-card-img` class purely so CSS can target it; the de-grayscale rules use `!important` to beat the inline `filter` on the element.
- **Stat counters** fire a small `scale(1.04)` yoyo pop (transform-origin left, so they grow in place beside the left-aligned label) via an `onComplete` on the GSAP count-up in `useScrollAnimations.ts`. The counter's `once: true` ScrollTrigger guarantees it fires only once per view.
- **Award arrows** already slid diagonally (`translate(2px, -2px)`) and glowed on hover before revision 4; this was left unchanged, as the diagonal reads as "opens externally" better than a flat slide would.

Part 10: Search visibility

Metadata

Every page has a title, a description, and Open Graph and Twitter card tags. The site-wide defaults live in `layout.tsx` with a template that appends "Dev Vyas" to each page title. Locale is set to `en_GB` and the document language to `en-GB`.

A social sharing image is generated at build time by `opengraph-image.tsx`.

Structured data

Machine-readable descriptions of each page, in JSON-LD:

- **Homepage** - person and site information
- **Case studies** - a breadcrumb trail plus a CreativeWork description
- **Vyas Graphics** - additionally, six VideoObject entries, one per real video, with exact durations
- **Blog posts** - BlogPosting schema with author, dates and keywords

The blog schema is generated once inside `BlogPostLayout` from the post data, rather than repeated by hand in each article.

Sitemap and robots

Both generated. The sitemap lists all seven routes with genuine last-modified dates rather than the build timestamp, which would otherwise tell search engines that every page changed every time the site was deployed.

Assets

Every image and video filename is descriptive and hyphenated, for example `icc-t20-world-cup-2026-india-champions-poster.jpg` rather than a numbered name.

Part 11: Accessibility

Work done deliberately:

- **Heading structure** - a single `h1` per page, with `h2` for sections and `h3` for items inside them. This was fixed after an audit found most sections jumping straight from `h1` to `h4`, and two sections having no semantic heading at all.
- **Zoom** - pinch-to-zoom is not blocked. The viewport previously capped maximum scale at 1, which fails WCAG 1.4.4 for anyone who needs to zoom in to read.
- **Reduced motion** - respected throughout. The welcome reveal collapses, transitions shorten, decorative animation stops.
- **Alt text** - every image has one; decorative icons paired with visible text are correctly given empty alt so screen readers do not announce them twice.
- **Focus states** - visible focus outlines follow each button's own shape.

- **Touch feedback** - every interactive element confirms a tap visually.
- **Switch semantics** - the illumine card toggles use `role="switch"` with `aria-checked`, so their state is announced rather than merely visible.
- **Contrast over drama** - the dimmed state of the tool cards was deliberately lightened from what looked best, to keep tool names above the 4.5:1 contrast floor.
- **Touch parity** - interactions that were hover-only in their source form (the flipbook cards, the Instagram button) are driven by state on touch devices, so nothing is inert on a phone.

Part 12: The contact form

Posts to `/api/contact`, which sends an email through Resend to `vyasdev.6303@gmail.com`. Input is trimmed and length-capped, and all user input is HTML-escaped before being placed in the email body.

The send button has a deliberate animation: the letters lift in a wave on hover, and on submission a paper plane takes off before the confirmation appears. The success panel is held back for 2.7 seconds so the animation finishes rather than being cut off mid-flight.

Environment variable required: `RESEND_API_KEY`.

Part 13: Things to be careful about

This section records problems that were genuinely difficult to diagnose. Most of them looked like one thing and turned out to be another.

Styling

Styles are defined twice: in the compiled CSS and in the SCSS source. `public/assets/css/styles.css` and `public/assets/scss/` both feed the build, so a value set in only one of them can be silently overridden by the other. This has now caught two separate changes: the background colour (`_themes.scss`) and the heading scale (`core/_typography.scss`). Both times the symptom was identical - the edit looked correct, the build succeeded, and the page did not change. If a style change appears to do nothing, grep the SCSS folder for the old value before assuming a caching problem.

Buttons are flex containers. Site-wide CSS sets `button { display: inline-flex }`, which makes `text-align: center` do nothing on a button. Use `justify-content` and `align-items` instead.

Mobile overrides may need `!important`. The build does not reliably preserve source order across media query boundaries, so a mobile-specific override can lose to a desktop rule that happens to be emitted later.

Check whether a rule targets a tag or a class before changing a tag. The heading hierarchy work was only safe because every affected style turned out to be class-based. A rule written as `h4.some-class` would have broken silently.

Layout and motion

Do not put a backdrop filter above a video. If `backdrop-filter` appears on an ancestor or sibling of a `<video>` element, the video does not render at all. This is a real browser behaviour and it cost a long debugging session on the Vyas Graphics page.

Use clip-path for the before-and-after slider. Updating width or position on every pointer move causes layout thrashing. `clip-path: inset()` is composited on the GPU and stays smooth.

`offsetTop` is not measured from the page. It is measured from the nearest positioned ancestor. Because `#wrapper` has `position: relative`, adding the welcome reveal above it silently broke every measurement based on `offsetTop`, by exactly the height of the welcome section. Use `getBoundingClientRect()` instead, which is always accurate relative to the viewport.

Inline styles beat CSS classes. The back-to-top button's press animation did nothing for a while because an inline `transform` (driving its show and hide) was silently overriding the class-based `transform` (driving the press). Only one mechanism can own a given property.

React re-renders overwrite direct DOM changes. A style set directly on an element will be wiped the next time React re-renders that element with a style object, even if the values look identical. The header clock ticking once a second was enough to cause this.

A fixed button will collide with scrolling content sooner or later. The mobile menu button sits at a fixed position 16px from the right edge and is 40px square, so it occupies that strip for the whole height of the page. The skills accordion put its own 40px circle hard against the same edge, so any time a header scrolled near the top the two overlapped and read as one broken control. Measured: both spanning x334 to x374 on a 390px viewport. Fixed by insetting the accordion circle on mobile only. Anything else placed against the right edge needs the same check.

Screenshots taken too early will show empty boxes. Images below the fold are lazy-loaded, so a screenshot fired immediately after scrolling an element into view can catch it before anything has painted, which looks exactly like a broken image. Before concluding an

image is failing, check `complete` and `naturalWidth` on the element, and whether any request actually 404'd. During this project that false alarm came up more than once.

An element with opacity below 1 creates a stacking context. This traps the z-index of any fixed-position children inside it, which is how the navigation sidebar ended up visible but unclickable, with the main content painting over it.

Animation

- Lenis must be configured with `syncTouch: false`.
- GSAP plugins are registered in exactly one place, `useScrollAnimations.ts`. Do not register them again elsewhere.
- The reduced-motion CSS block must not disable the discipline marquee or the scribble drawing.
- Never add a static `transform` to the circular badge; it conflicts with the spin animation's own transform and freezes it.
- The work section's active-card tracking must read live positions on every scroll tick. Two earlier approaches using GSAP callbacks and `offsetTop` both failed.

Writing and content

- No em dashes or en dashes anywhere. Use a plain hyphen.
- British English throughout: colour, organise, behaviour, judgement, recognise, analyse.
- Conversational but professional. Avoid mirroring a CV too literally.
- The dissertation content stays high level until submission.

Part 14: Making changes

Common updates

Changing a job title, description or date - edit the relevant file in `src/data`. Nothing else needs touching.

Adding a certification - add an entry to `awards.ts`.

Adding a timeline entry - add to `educationItems` in `education.ts`, keeping chronological order.

Adding a tool - add to the appropriate category in `tech.ts`. If the tool's logo is pure black it will be invisible on the dark background, so recolour the SVG to white first.

Writing a new article - add an entry to `blog.ts`, create `src/app/blog/[slug]/page.tsx` using `BlogPostLayout`, add the ISO date to the lookup in `BlogPostLayout.tsx`, and add the route to `sitemap.ts`.

Changing the accent colour - search for `#00DE51` and `#00C853` across `styles.css` and components.

Deployment

The site deploys automatically from the `main` branch on Vercel.

When applying an updated project archive, extract it into a staging folder and merge rather than replacing the project folder directly:

```
# Extract the archive into Website Version/zip/
rsync -a --exclude='.git' zip/dev-nextjs/ dev-nextjs/
cd dev-nextjs
git add .
git commit -m "Describe the change"
git push origin main
```

Never run `git init` in the project folder, and never force-push. The repository holds a long commit history and both carry a real risk of losing it.

Local development

```
npm install
npm run dev      # development server
npm run build    # production build
npm run start    # serve the production build
```

If a change does not appear after a rebuild, delete the `.next` folder and check that no old server process is still running. Several apparent bugs during development turned out to be a stale process serving old files.

Part 15: What is still open

Custom domain. The remaining infrastructure task. The site currently runs on its Vercel subdomain.

Dissertation results. Once the dissertation is submitted, the case study and the related article can be updated with exact statistics and findings.

Optional cleanup. `BentoIntro.tsx` and `BrandSlider.tsx` are unused and could be removed if you are confident they will not be wanted. The Tomorrow ExtraBold font file is also no longer referenced, now that the welcome title sits at weight 700.

Deployment logos. React, GitHub, Vercel and Firebase now come from the official brand files. Two needed work before they could be used: the Vercel and Firebase files supplied were full wordmarks, so their viewBoxes were cropped to the triangle and the flame respectively, since the pill already prints the tool's name beside the icon. GitHub and Vercel are both drawn in white, as their brand black would be invisible here.

Netlify is the exception. The official file for it is a wordmark whose letterforms are the logo, with no separable icon inside it, so the icon mark already in place was kept. If a square Netlify icon file turns up later it can be dropped straight in over `tech-netlify.svg` without touching any code, since the filename is what `tech.ts` points at.

Appendix: Quick reference

Key files

File	What it controls
`src/data/profile.ts`	Name, title, bio, email, availability, social links
`src/data/works.ts`	The three project cards
`src/data/blog.ts`	Article metadata
`src/data/skills.ts`	The four skill areas
`src/data/tech.ts`	Tool categories
`src/data/education.ts`	Background timeline
`src/data/awards.ts`	Certifications
`src/data/nav.ts`	Navigation items
`src/app/layout.tsx`	Site-wide metadata
`src/app/sitemap.ts`	Sitemap and last-modified dates
`public/assets/css/styles.css`	Nearly all styling

Contact details on the site

- Email: vyasdev.6303@gmail.com
- LinkedIn: linkedin.com/in/dev-vyas6
- Instagram: instagram.com/vyas.graphics
- Behance: behance.net/devvyas_graphics
- Location: York, United Kingdom
- Availability: from September 2026, UK and remote

End of document.